

Programming Fundamentals (COSC2531)

Assignment2: Deliverable 1

Assessment Type	Individual assignment (no group work). Submit online via Canvas/Assignments/Assignment 2: Deliverable 1. Marks are awarded per rubric (please see the rubric on Canvas). Clarifications/updates may be made via announcements. Questions can be raised via the lectorial, practical sessions or Canvas discussion forum. Note the Canvas discussion forum is preferable.
Due Date	End of Week 10 (exact time is shown in Canvas/Assignments/Assignment 2) Deadline will not be advanced nor extended. Please check Canvas/Assignments/Assignment 2: Milestone 1 for the most up to date information regarding the assignment. As this is a major assignment, a University standard late penalty of 10% per day applies (i.e., 1 mark/day), unless extension request was approved by the course coordinator or special consideration has been granted by the school.
Weighting	10 marks out of 100
Version	Released 8/09/2026. Please treat this document as draft. Minor paraphrasing and adjustments might be added to improve clarity. Please report any issue by 11/09/2026, after this date the spec will be considered final.

1. Overview

The main objective of this assignment is to familiarize you with object-oriented design and programming. Object-oriented programming helps to solve complex problems by coming up with several domain classes and associations. However, identifying meaningful classes and interactions requires a fair amount of design experience. Such experience cannot be gained by classroom-based teaching alone but must be gained through project experience. This assignment is designed to introduce OOP's core principles: Abstraction, Polymorphism, Inheritance, and Encapsulation (APIE).

You should develop this assignment in an iterative fashion (as opposed to completing it in one sitting). You can and should get started now (when this assignment specification is posted on Canvas) as there are concepts from previous lessons that you can employ to do this assignment. If there are questions, you can ask via the lectorial, practical sessions or the Canvas discussion forum (Canvas/Discussions/Discussion on Assessment 2). Note that the **Canvas discussion forum is preferable** as it allows other students to see your questions as well. Also, you should ask questions in a general manner, for example, you should replicate your problem in a different context in isolation before posting, and you must not post your code on the Canvas discussion forum.

2.AssessmentCriteria

This assignment will determine your ability to:

- i. Follow coding, convention and behavioural requirements provided in this document and in the course lessons;
- ii. Independently solve a problem by using programming concepts taught over the duration of the course;
- iii. Write and debug Python code independently;
- iv. Document code;
- v. Provide references where due;
- vi. Meet deadlines;
- vii. Seek clarification from your "supervisor" (instructor) when needed via the Canvas discussion forums; and;
- viii. Create a program by recalling concepts taught in class, understand, and apply concepts relevant to solution, analyse components of the problem, evaluate different approaches.

3. Learning Outcomes

This assignment is relevant to the following Learning Outcomes:

1. Analyse simple computing problems.
2. Devise suitable algorithmic solutions and code these algorithmic solutions in a computer programming language.
3. Develop maintainable and reusable solutions using the object-oriented paradigm.

Specifically, upon the completion of this assignment, you will be able to:

- Demonstrate knowledge of basic concepts, syntax, and control structures in programming.
- Devise solutions for simple computing problems under specific requirements with the object-oriented programming paradigm.
- Encode the devised solutions into computer programs and test the programs on a computer.
- Demonstrate understanding of standard coding conventions and ethical considerations in programming.

4. Assessment Details

Board Game Café OOP System

You work for a café that lends board games to tables *and* serves food and drink from a kitchen. Both are tracked through one system, per table, so a customer's session might include a borrowed game, a round of prep, and a bill covering both. The system must be extendible: each component needs to be modular, encapsulated, and reusable as much as possible.

Assignment 2 project comprises three deliverables:

- A2 Deliverable 1 MVP Code — 10% (D1)
- A2 Deliverable 2 Final code — 15% (D2)

- A2 Deliverable 3 Demo and interview — 15% (D3) based on your D1 and D2 experience and submissions.

You will start with a proof-of-concept minimum viable product (MVP) for **D1**, built into a prescriptive spec so you meet client expectations. **D2** refactors and extends D1 with greater complexity, requiring deeper application of OOP's core principles: **A**bstraction, **P**olymorphism, **I**nheritance, and **E**ncapsulation (APIE).

This assessment specifies the MVP requirements which serve as the first milestone check for Assignment 2 whose project description is available on Canvas.

In this milestone, you are required to demonstrate your progress towards completing Assignment 2.

Requirements gathering and analysis

From the requirements gathering and analysis stage of the project, the following key business processes were noted:

- A group of customers would start a game session by finding a table, borrowing a board game, and ordering consumables for unlimited time for a per-guest flat fee.
- A cafe employee would:
 - Keep track of what each table has borrowed and ordered.
 - Keep track of running total spend or hire (running tab) by each table.
 - Add any kind of offering — a game or a consumable (food/drink item) to customer table's session.
 - Have any offering "actioned" (a game set up and ready to play; a food item prepared).
 - Bill the customer.
 - Save a table's session for a repeat visit (a regular's standing order/favourite game).

Based on these facts, the cafe requires combining two quite different offerings (a physical, reusable board game vs. a consumed, prepared food/drink item) under one system, which forces a genuine abstraction decision rather than two classes that just happen to share a name and a price.

As a starting point, it has been decided that the cafe employee will be the only user of this system. Any order and transaction by a customer will be captured in this system, however the employee will be the only authorised party to interact with the system.

Your work will be assessed based on two main criteria:

- Code implementation, based on the specified behavioural requirements.
- Core functionalities.

CODE IMPLEMENTATION (5 marks)

This spec does **not** name the classes for you. Deciding what's shared between a board game and a food item, what genuinely differs, and where that boundary sits **is** the task. Be ready to justify your design in the code documentation and D3 interview. You need to model appropriate class structures to capture the right data

(attributes) and behaviours (methods / functions). The best approach would be to map each value in the CSV files with the relevant class attributes. While some of the attributes may not be involved in the operations of the MVP this assignment, they may be used in later, final deliverable.

Task 1 – Customer class, a model for customer data and behaviours is included in the provided template code. Investigate how this should relate to the provided customers.csv and how it should fit and interact with the rest of code that you are implementing. Make modifications as required.

Task 2 - Model an offering. Design a representation of something a table can add to its session that captures at minimum: name, price, and a 'service time'. At least two categories must exist — a **board game** (physical, borrowed and returned, has a player-count range and a difficulty/complexity level) and a **food/drink item** (consumed, prepared, has a prep time and — where relevant — a spice or temperature setting). These two categories must share a common interface but genuinely diverge in behaviour: e.g. "servicing" a game means checking it out and eventually checking it back in, while "servicing" a food item means preparing it once and it's gone. If your two subclasses only differ by a label, your abstraction is too shallow – consider reworking it.

Task 3 - Model the catalogue. Design a component responsible for loading the available games and menu items from the relevant CSV file and making them queryable by name, category, or price. It should not expose its internal storage structure directly to other components.

Task 4 – Model a table configuration. Design a component that holds information about table configuration, e.g. to identify where each table is situated in the café how many guests can it accommodate.

Task 5 - Model a table's session. Design a component that holds everything a specific table has added - games and food together - supports adding and removing offerings and can compute a running bill that correctly handles a game's hire fee differently from a food item's one-off price. It should support being saved and reloaded (see Task 5).

Task 6 - Model servicing. Design a component that "services" an offering or a whole session, one item at a time, in a way that distinguishes checking a game out (and eventually back in) from preparing a food item once. Structure this so real timing can be added later without restructuring the rest of the system.

Task 7 - Persistence. Implement saving and loading of a table's session using CSV only. A saved session - including which games are currently checked out — must be able to be reloaded in a later sitting. Because of

Task 8 - Billing. Implement a way to display a table's current bill, itemised by offering, correctly totaling mixed game-hire and food charges.

Task 9 - CLI menu system. Build a command-line interface tying the above together: view catalogue, add offering, remove offering, service offering(s), save session, show bill, exit-and-save.

Task 10 - (Advanced) Recommender function. This is to scan the current session situation on each table and recommend the most suitable table, game, and for a particular customer in their upcoming session. This recommendation will be based on the following factors:

- Current availability and capacities of tables
- Saved session preference for game (group size, difficulty, availability)

For example, Bruce and his three friends occupied a table of four, played Catan which is a competitive, cooperative, Euro, strategy board game for 3-4 players with around medium difficulty. Based on this retained knowledge about the customer, the system recommends the same or similar game and table configuration for the next session.

All tasks are not optional however your decision to use GenAI is optional. Particularly for Task 8 you may use Val's assistance for brainstorming, getting advice on structure and syntax, and designing the most efficient algorithm to solve this problem. Your skills in using GenAI tool will not be assessed, however your reasoning, justification and understanding of your own code implementation will be examined in D3 – interview.

Few known constraints, MVP limitations and exclusions:

- A table can be allocated to only one customer during a session. Once occupied, it cannot be assigned to another customer until it has been released.

- A customer may bring any number of guests, provided the table has sufficient capacity. Only the primary customer needs to be registered; the names of the other guests are not required. The registered customer is responsible for paying the bill.
- Services can be added as many times as required per session until the customer checks out. Upon checking out, the game will be returned, the tab will be settled, the bill will be presented, and the table will be available for other customers.
- Each session is bookable for a cover charge which is a flat fee per guest and will cover the cost of borrowing as many as two board games, one game at a time, which implies that the first game must be returned before the second game can be borrowed. At this stage, once the two games limit has been reached, the café will just refuse to hand out any more games when asked, however each game can be played as many times as desired.
- The café at this stage only has 1 stock of each board game.
 - Recommendation by game genre may be required at later stage, but not this stage.
 - Consumables are not required to be tracked or saved in session file.
 - The following CSV data are not required to be used in program operation in this MVP stage, however, feel free to still map them as class attributes as placeholders for future stage:
 - Tables.csv: HasPowerOutlet
 - Offerings.csv: PrepTime (Mins)
 - Customers.csv: Membership Type, Budget, Loyalty Points

FUNCTIONALITY (4 marks)

This component addresses whether your code implementation is functional. For this milestone check (A2 D1), at a minimum, your code is expected to have the following functionalities:

1. When the program starts, it looks for the files *customers.txt* (the customer file), *offerings.txt* (contains catalog of services to be offered to customers), *tables.txt* (table configuration), and *sessions.txt* (keep track of each session) in the local directory (the directory that stores the .py file of the program), read the data from these files into the program accordingly. Then the program will then display a menu with the 7 options described below. In this milestone check, it is assumed that there will always be these 3 files in the local directory with the right formatting. Note that in this milestone check, service packages are not yet required.
2. The menu option to *Allocate customers to a session*, bearing in mind that group of customers for each session is represented by just 1 customer by either ID or name.
3. The menu option to *Add an offering to a table's session*.
4. The menu option to *Remove an offering from a table's session*.
5. The menu option to *Simulate the servicing of an offering*, for example:
 1. Setting up a game – simply show some captions describing what's going on.
 2. Preparing a food item – simply show some captions describing what's going on.

Hint: Use `time.sleep()` function to create half a second animated delays between activities, i.e. each caption should be shown for 0.5 seconds before the next one appears.

6. The menu option *Show the current bill* will be able to display the services that's been provided to a session.
7. The menu option to *Recommend allocation*, which will provide offering recommendation based on customer group size and past session game preferences, and if accepted; the same effect as running Option 2 and 3 will be achieved. *Hint:* consider the re-usability aspect of objects and methods.
8. The menu option *Exit the program* will allow users to exit the program while saving the configuration of each table session.

Other Requirements:

The program **must be entirely in one Python file named `ProgFunA2_D1_Your Student ID.py`**. For example, if your student ID is `s1234567`, then the Python file must be named `ProgFunA2_D1_s1234567.py`. Other names will not be accepted. Program implementation in other language besides Python will not be eligible for any mark. Implementing the program using a paradigm other than Object-Oriented Programming may result in the submission being rejected or heavily penalised.

Please implement a reasonable input validation and error handling, especially in the parts of the program that gets input from the user and data files. A crashing and buggy program will attract some penalties.

Mandatory reflection log (1 mark)

Program documentation and design justification must be written as code comments in the Python file you submitted above.

At the beginning of your Python file, in the code comments, your code must contain the following information: **your name and student ID**.

Write a short paragraph that:

- Names a specific OOP principle that you have used – especially how you handled the game / food divergence in Task 1 – what alternative you considered and why you chose one over the other. For example, why did you choose inheritance over composition, or vice versa. Another example, where and how you implemented encapsulation and why that part is particularly benefited from it.
- Reference to a specific bug you encountered, describes the symptom, and explains exactly how you diagnosed and fixed it.

You must demonstrate your ability to program in Python by yourself, i.e., you should not attempt to use external special Python packages/libraries/classes that can do most of the coding for you. A program fully implemented by Gen AI will not be worth any mark. **The only Python libraries allowed in this assignment are `sys`, `datetime`, `os`, and `time`.**

Because of the nature of supplied data structure in the provided CSV files, which requires careful handling of the delimiters, the built-in `csv` module or any other 3rd party library is not allowed.

Your code should document the references, i.e., any sources of information (e.g., websites) you used other than the course contents directly under Canvas/Modules, **you must give acknowledgement of the sources, explaining how you use the sources in this assignment**. More detailed information regarding the references can be found in Section 7.

Note that in places where this specification may not tell you how exactly you should implement a certain

feature, you need to use your judgment to choose and apply the most appropriate concepts from our course materials. You should follow answers given by your "client" (or "supervisor" or the teaching team) under Canvas/Discussions/Discussion on Assessment 2.

Besides the program file, you must submit a GenAI framework Condition 3 declaration form in pdf format.

Mark Allocation:

Assessment Component	Points
Code Implementation	5
Functionality	4
Code documentation	1
Declaration of GenAI Condition 3	No mark, but must be provided.

More details of the rubric of this assignment can be found on Canvas. Students are required to look at the rubric to understand how the assignment will be graded.

Future consideration:

In Assignment 2 Deliverable 2 (D2), you'll extend the system with additional features and greater complexity, which will require refactoring your current code. If it's well designed, this extension should be straightforward. The full D2 specification will be released at a later stage.

5. Submission

As mentioned in the Other Requirements, **you must submit only two files:**

- **ProgFunA2_D1_YourStudentID.py**, containing all your program source code including all classes and program documentation.
- **Condition3_Bounded_Process_AI_SubmissionDoc_YourStudentID.pdf**

via Canvas/Assignments/Assignment 2: Deliverable 1. **It is your responsibility to correctly submit your files.** Please verify that your submission is correctly submitted by downloading what you have submitted to see if the file includes the correct contents. The final .py file submitted is the one that will be marked.

Please do not submit any CSV or any other data files because your program will be tested using the provided CSV files and our own test files.

Please do not bundle your files in a zip your submission as this will hinder plagiarism scanner and slow down the marking process. Any submission bundled in a zip or any other format will get penalised.

Late Submission

All assignments will be marked as if submitted on time. Late submissions of assignments without special consideration or extension will be automatically penalised at a rate of 10% of the total marks available per day (or part of a day) late. For example, if an assignment or assignment component is worth 10 marks and it is submitted 1 day late, a penalty of 10% or 1 mark will apply. This will be deducted from the assessed mark.

Assignments will not be accepted if more than five days late unless special consideration or an extension of time has been approved.

Special Consideration

If you are applying for extensions for your assessment within five working days after the original assessment date or due date has passed, or if you are seeking extension for more than seven days, you will have to apply for Special Consideration, unless there are special instructions on your Equitable Learning Plan.

In most cases you can apply for special consideration online [here](#). For more information on special consideration, visit the university website on special consideration [here](#).

6. Referencing Guidelines

What: This is an individual assignment, and all submitted contents must be your own. If you have used any sources of information (e.g., websites, tools) other than the course contents directly under Canvas/Modules, **you must give acknowledgement of the sources, explaining in detail how you use the sources in this assignment, and give references using the [IEEE referencing format](#).**

Where: You can add a code comment near the work (e.g., code block) to be referenced and include the detailed reference in the IEEE style.

How: To generate a valid IEEE style reference, please use the [citethisforme](#) tool and RMIT Library [Easy Cite](#) if you're unfamiliar with this style.

7. Academic Integrity and Plagiarism (Standard Warning)

Academic integrity is about honest presentation of your academic work. It means acknowledging the work of others whilst developing your own insights, knowledge, and ideas. You should take extreme care that you have:

- Acknowledged words, data, diagrams, models, frameworks and/or ideas of others you have quoted (i.e., directly copied), summarized, paraphrased, discussed, or mentioned in your assessment through the appropriate referencing methods.
- Provided a reference list of the publication details so your readers can locate the source if necessary. This includes material taken from internet sites.

If you do not acknowledge the sources of your material, you may be accused of plagiarism because you have passed off the work and ideas of another person without appropriate reference, as if they were your own.

RMIT University treats plagiarism as a very serious offence constituting misconduct. Plagiarism covers a variety of inappropriate behaviors, including:

- Failure to properly document a source
- Copyright material from the internet or databases
- Collusion between students

For further information on our policies and procedures, please refer to the University website ([link](#)).

8. Assessment Declaration:

When you submit work electronically, you agree to the assessment declaration:
<https://www.rmit.edu.au/students/student-essentials/assessment-and-results/how-to-submit-your-assessments>

GenAI Framework Condition 3 applies to this assignment, which means that you may use the RMIT-endorsed GenAI tool, Val, as a learning assistant within a controlled environment to address the presented challenges within defined boundaries. No other Gen AI tool is allowed on this course, for the following reasons:

- Firstly, because we want you to learn the fundamentals by understanding how codes and programs work at a deeper reasoning level, so that at later stage of your studies and career you will be able to independently criticise and improve incorrect programs.
- Secondly, we want everyone's learning experience fair and equitable, because by using the same AI tool we will be able to limit the discrepancies and gaps in recommendations generated by various tools.
- Ensure relevance with the key topics being taught.

Regardless of your decision to use or not use Gen AI, you must fill out and submit the Gen AI Framework Condition 3 declaration form.